

## Diseño y Arquitectura

El desarrollo del simulador se realizó en C++ utilizando el framework Qt. Para garantizar un código escalable y eficiente, se tomaron las siguientes decisiones de diseño:

- **Separación de Lógica e Interfaz:** Se dividió el proyecto estrictamente en el Motor Lógico (Planificador.cpp y Proceso.h) y la interfaz gráfica (mainwindow.cpp). El planificador es completamente ajeno a la interfaz gráfica.
- **Gestión del Tiempo mediante QTimer:** En lugar de utilizar bucles while tradicionales que bloquearían el hilo principal de la aplicación, se implementó un QTimer en Qt. Este emite una señal asíncrona cada segundo, invocando la función ejecutarPaso() para avanzar la simulación un tick de procesador, permitiendo la actualización fluida de la GUI.
- **Optimización de Estructuras de Datos:** Para representar las colas de Listos y Bloqueados, se utilizaron vectores que almacenan únicamente el identificador del proceso.

## Desafíos de Implementación

Durante el desarrollo del simulador, surgieron varios retos de sincronización y modelado:

1. **El Orden de Ejecución del Tick:** Inicialmente, el simulador ejecutaba la ráfaga del proceso actual antes de asignar la CPU a uno nuevo. Esto causaba que, en el instante en que la CPU quedaba libre, se perdiera un ciclo de reloj completo. La solución fue reestructurar ejecutarPaso() en un orden estricto: Llegadas- Asignación de CPU-Ejecución del Tick-Verificación de E/S.
2. **Ciclo de Vida y Procesos Zombies:** Al modelar las operaciones de Entrada/Salida, el pensamiento inicial fue devolver el proceso a la cola de Listos tras terminar su bloqueo. Sin embargo, dado que el modelo establece una única ráfaga de CPU seguida de una ráfaga de E/S, devolverlo generaba procesos zombies con 0 tiempo restante que distorsionaban el tiempo promedio de espera. Se corrigió enviando los procesos directamente al estado Terminado al finalizar su E/S.
3. **Fase de Expulsión:** La implementación de algoritmos expulsivos (como Round Robin y SRTF) requirió dividir la lógica de planificación para poder evaluar, en cada *tick*, si el proceso en ejecución debía ser desalojado antes de que terminara su ráfaga natural.

## Análisis de Rendimiento

Para este análisis, se generó un lote fijo de 5 procesos con ráfagas, tiempos de llegada, prioridad y E/S aleatorios. El lote se sometió a tres algoritmos distintos para observar su comportamiento.

- **Primero en llegar, primero en ejecutar (FCFS):**
  - **Tiempo Total:** 20 ticks
  - **Total procesos completados:** 5
  - **Uso del Procesador:** 95.00 %
  - **Tiempo promedio de espera:** 7.40 ticks

- **Tiempo promedio de ejecución (Retorno):** 11.40 ticks
- **Tiempo promedio en bloqueo (E/S):** 0.60 ticks
- **Arribo de procesos por paso:** 0.250 procesos/tick
- FCFS demostró ser el algoritmo más ineficiente en términos de tiempo de espera debido al conocido Efecto Convoy. Los procesos cortos que llegaron después de un proceso con una ráfaga larga tuvieron que esperar pasivamente, elevando el promedio global.
- **Primero el trabajo más corto (SJF):**
  - **Tiempo Total:** 19 ticks
  - **Total procesos completados:** 5
  - **Uso del Procesador:** 100.00 %
  - **Tiempo promedio de espera:** 6.60 ticks
  - **Tiempo promedio de ejecución (Retorno):** 10.60 ticks
  - **Tiempo promedio en bloqueo (E/S):** 0.60 ticks
  - **Arribo de procesos por paso:** 0.263 procesos/tick
  - Como establece la teoría, SJF demostró ser el algoritmo óptimo para minimizar el tiempo de espera. Al reordenar la cola dando prioridad a las ráfagas más cortas, el flujo de finalización fue mucho más rápido.
- **Turno Rotativo (Round Robin - Quantum 3):**
  - **Tiempo Total:** 19 ticks
  - **Total procesos completados:** 5
  - **Uso del Procesador:** 100.00 %
  - **Tiempo promedio de espera:** 9.40 ticks
  - **Tiempo promedio de ejecución (Retorno):** 13.40 ticks
  - **Tiempo promedio en bloqueo (E/S):** 0.60 ticks
  - **Arribo de procesos por paso:** 0.263 procesos/tick
  - Round Robin presentó un tiempo de espera y retorno ligeramente mayor que SJF, pero garantizó equidad. Se pudo observar en tiempo real cómo la CPU alternaba su atención, demostrando que ningún proceso sufrió de inanición. Es el algoritmo ideal para sistemas que requieren alta interactividad y tiempos de respuesta rápidos para el usuario.
- **Primero el menor tiempo restante (SRTF):**
  - **Tiempo Total:** 19 ticks
  - **Total procesos completados:** 5
  - **Uso del Procesador:** 100.00 %
  - **Tiempo promedio de espera:** 5.00 ticks
  - **Tiempo promedio de ejecución (Retorno):** 9.00 ticks
  - **Tiempo promedio en bloqueo (E/S):** 0.60 ticks
  - **Arribo de procesos por paso:** 0.263 procesos/tick
  - Al ser la versión expulsiva de SJF, SRTF optimizó aún más el tiempo promedio de espera frente a la llegada de nuevos procesos cortos. Sin embargo, empíricamente se nota que esto tiene un costo: genera un mayor número de cambios de contexto (interrupciones de la CPU), lo que en un sistema operativo real se traduciría en una mayor sobrecarga del sistema.
- **Planificación basada en Prioridades (No Expulsivo):**
  - **Tiempo Total:** 19 ticks
  - **Total procesos completados:** 5
  - **Uso del Procesador:** 100.00 %

- **Tiempo promedio de espera: 6.80 ticks**
- **Tiempo promedio de ejecución (Retorno): 10.80 ticks**
- **Tiempo promedio en bloqueo (E/S): 0.60 ticks**
- **Arribo de procesos por paso: 0.263 procesos/tick**
- El rendimiento global de este algoritmo es muy variable, ya que ignora por completo la longitud de las ráfagas. Garantiza que los procesos críticos (mayor prioridad) se ejecuten pronto, pero durante la simulación es evidente su mayor defecto teórico, la inanición. Los procesos con prioridad baja acumulan tiempos de espera masivos si siguen llegando procesos con mayor prioridad.
- **Planificación basada en Prioridades (Expulsivo):**
  - **Tiempo Total: 19 ticks**
  - **Total procesos completados: 5**
  - **Uso del Procesador: 100.00 %**
  - **Tiempo promedio de espera: 7.20 ticks**
  - **Tiempo promedio de ejecución (Retorno): 11.20 ticks**
  - **Tiempo promedio en bloqueo (E/S): 0.60 ticks**
  - **Arribo de procesos por paso: 0.263 procesos/tick**
  - Mejora drásticamente el tiempo de respuesta inicial para los procesos de alta prioridad, ya que ocupan la CPU en el instante en que llegan a la cola de listos. No obstante, las métricas globales de espera para el lote completo suelen ser peores que en algoritmos enfocados en el tiempo (como SJF), penalizando severamente a los procesos menos importantes.
- **Selección Aleatoria (Random):**
  - **Tiempo Total: 19 ticks**
  - **Total procesos completados: 5**
  - **Uso del Procesador: 100.00 %**
  - **Tiempo promedio de espera: 7.40 ticks**
  - **Tiempo promedio de ejecución (Retorno): 11.40 ticks**
  - **Tiempo promedio en bloqueo (E/S): 0.60 ticks**
  - **Arribo de procesos por paso: 0.263 procesos/tick**
  - Como era de esperarse, este algoritmo presentó métricas impredecibles y generalmente deficientes. Su inclusión en el simulador sirve como un grupo de control. Al comparar sus resultados caóticos con las métricas ordenadas de los demás algoritmos, se comprueba matemáticamente el valor y la necesidad imperativa de aplicar políticas de planificación lógicas en un sistema operativo.

## Conclusión

El desarrollo de este simulador permitió comprender la complejidad que enfrenta el planificador a corto plazo de un sistema operativo. Se demostró que no existe un algoritmo perfecto, sino que cada uno ofrece un compromiso entre el uso del procesador y el tiempo de espera.